DOCUMENTAÇÃO TÉCNICA

Acompanhamento Diário

Aplicação web para registro de atividades e acompanhamento do apontamento de horas em ciclos semanais de sexta a quinta-feira.

VERSÃO
0.1.0

DATA
26/10/2026

REPOSITÓRIO
peubibytes/acompanhamento_diario

React 19 · TypeScript · Vite 8 · Tailwind CSS 4 · TanStack Query 5 · React Router 8 · Node.js ≥ 23.6 · Express 5 · SQLite (node:sqlite) · Zod 4

Sumário

1 **Visão geral** — Propósito, conceitos e escopo
2 **Regras de negócio** — Ciclo, meta, métricas e conformidade
3 **Arquitetura** — Monorepo, camadas e decisões técnicas
4 **Pacote `shared`** — Domínio compartilhado
5 **Back-end** — Persistência, repositório e servidor HTTP
6 **Referência da API REST** — Endpoints, contratos e erros
7 **Front-end** — Telas, estado, componentes e tema
8 **Execução e operação** — Instalação, scripts e configuração
9 **Qualidade** — Testes, tipos e convenções
10 **Roadmap** — Evoluções planejadas

Documento gerado a partir de `docs/fonte/documentacao.html`.

1 Visão geral

O que a aplicação resolve, os conceitos do domínio e o que está dentro e fora do escopo.

1.1 Propósito

O **Acompanhamento Diário** substitui uma planilha Excel de controle de horas por uma aplicação web. O usuário registra, ao longo da semana, as atividades realizadas, com horário de início e fim. A aplicação consolida esses registros em métricas diárias e semanais e indica, a cada momento, quanto da carga horária do ciclo já foi apontada.

O objetivo operacional é chegar à **quinta-feira com 100% das horas preenchidas**. Nesse dia, o usuário transfere os apontamentos para um sistema externo (Azure DevOps). Essa integração está fora do escopo da versão atual.

1.2 Conceitos do domínio

TERMO	DEFINIÇÃO
Ciclo	Período de apuração que começa na sexta-feira e termina na quinta-feira seguinte. Contém 5 dias úteis: Sex, Seg, Ter, Qua e Qui.
Atividade	Registro de trabalho em um dia útil: intervalo de horário, projeto/demanda, descrição, observações e status.
Meta diária	Carga horária esperada por dia, definida pelo usuário: 5h30, 8h48 ou um valor personalizado (“Outro”).
Meta semanal	Meta diária × 5 dias úteis.
Horas apontadas	Soma das durações das atividades de um dia ou do ciclo.
Conformidade	Situação do dia em relação à meta: *Conforme* (exatamente a meta), *Não conforme* ou *Pendente* (sem apontamento).
Irregularidade	Dia com horas acima da meta. É não conforme e exige reajuste do apontamento.
% preenchido	Horas apontadas no ciclo ÷ meta semanal.

1.3 Escopo da versão 0.1.0

Incluído

- CRUD de atividades por dia útil
- Métricas diárias e semanais equivalentes às fórmulas da planilha
- Meta diária configurável e versionada por ciclo
- Detecção e sinalização de dias irregulares
- Painel visual, registro por dia e tabela detalhada
- Temas claro e escuro, layout responsivo

Fora do escopo

- Integração com o Azure DevOps (códigos de work item, envio automático)
- Autenticação e múltiplos usuários. A aplicação roda localmente, para um único usuário.
- Lançamentos em sábados e domingos

2 Regras de negócio

Todas as regras ficam no pacote `shared` e são consumidas igualmente pelo servidor e pela interface.

2.1 Ciclo sexta → quinta

O ciclo de qualquer data é derivado dela mesma, sem tabela de semanas. A data de início é a sexta-feira mais recente que seja menor ou igual à data de referência:

```
offset = (diaDaSemana(data) - 5 + 7) mod 7      // 0 = domingo … 6 = sábado; 5 = sexta
inicio = data - offset dias                      // sexta-feira
fim    = inicio + 6 dias                         // quinta-feira
diasUteis = inicio + [0, 3, 4, 5, 6]             // Sex, Seg, Ter, Qua, Qui
```

Sábado e domingo pertencem ao ciclo da sexta-feira anterior, mas não são dias úteis e não aceitam lançamentos. As datas trafegam como texto ISO `AAAA-MM-DD`, e a aritmética é feita em UTC para evitar desvios de fuso horário e de horário de verão.

2.2 Meta diária

- **Opções:** 5h30 (330 min), 8h48 (528 min) ou "Outro", informado em `hh:mm` entre 1 min e 24h.
- **Versionamento por ciclo:** cada ciclo guarda a sua meta (`cycle_goals`). Sem registro próprio, o ciclo usa a meta padrão (`settings.default_daily_goal_minutes`, inicialmente 330).
- **Alteração no ciclo atual ou futuro:** a nova meta vale para o ciclo e passa a ser o padrão dos próximos. Antes da troca, os ciclos anteriores que tinham atividades e ainda seguiam o padrão antigo têm a meta congelada, para que o histórico não seja recalculado.
- **Alteração em ciclo passado:** afeta somente aquele ciclo.

2.3 Atividade

CAMPO	TIPO	REGRA
`date`	ISO date	Data válida e dia útil (segunda a sexta).
`startMinute`	inteiro	Minutos desde 00:00, de 0 a 1439.
`endMinute`	inteiro	De 1 a 1440, estritamente maior que `startMinute`.
`project`	texto	Obrigatório, de 1 a 200 caracteres (após *trim*).
`description`	texto	Opcional, até 2000 caracteres.
`notes`	texto	Observações/entregáveis, opcional, até 2000 caracteres.
`status`	enum	`concluido`, `nao_concluido`, `pendente_aprovacao` ou `nao_iniciado`.

A **duração** não é persistida: é sempre calculada como `endMinute - startMinute`. Horários sobrepostos no mesmo dia são aceitos, mas geram avisos (`warnings`) na resposta da API.

2.4 Métricas por dia

As métricas reproduzem as fórmulas da planilha original (colunas E a I). Considere `m` = minutos apontados e `g` = meta diária do ciclo:

MÉTRICA	REGRA	EXEMPLO (G = 5H30)
Horas apontadas	Σ durações das atividades do dia	`345` min
Diferença	vazio se `m = 0`; `00:00` se `m = g`; senão `±hh:mm`	`+00:15`
Qtd. atividades	atividades do dia com duração > 0	`4`
Conformidade	`pendente` se `m = 0`; `conforme` se `m = g`; senão `nao_conforme`	`nao_conforme`
Excedido	`m > 0` e `m > g`	`true`
Diagnóstico	"Sem apontamento" · "Meta 5h30 atingida" · "Faltam hh:mm" · "Excedeu a meta de 5h30 em hh:mm. Reajuste o apontamento"	"Excedeu a meta de 5h30 em 00:15. Reajuste o apontamento"

2.5 Métricas do ciclo

MÉTRICA	REGRA
Meta semanal	`g × 5`
Total realizado	Σ minutos dos 5 dias úteis
Dias conformes	quantidade de dias com `conforme`, exibida como `X/5`
Dias excedidos	quantidade de dias irregulares
% preenchido	`floor(total × 100 ÷ metaSemanal)`. O arredondamento para baixo garante que 100% só apareça com a semana realmente completa.
Situação geral	“100% conforme / Meta total atingida” se os 5 dias forem conformes; senão “X/5 conformes / Ajustes pendentes”

Regra de conformidade estrita. Somente o valor exato da meta é conforme. Excesso não compensa falta: um ciclo com 100% de horas distribuídas de forma desigual continua irregular. A interface só sinaliza “pronto para apontar” quando os 5 dias estão conformes.

3 Arquitetura

Monorepo com três pacotes npm (workspaces), um servidor HTTP único e banco SQLite em arquivo.

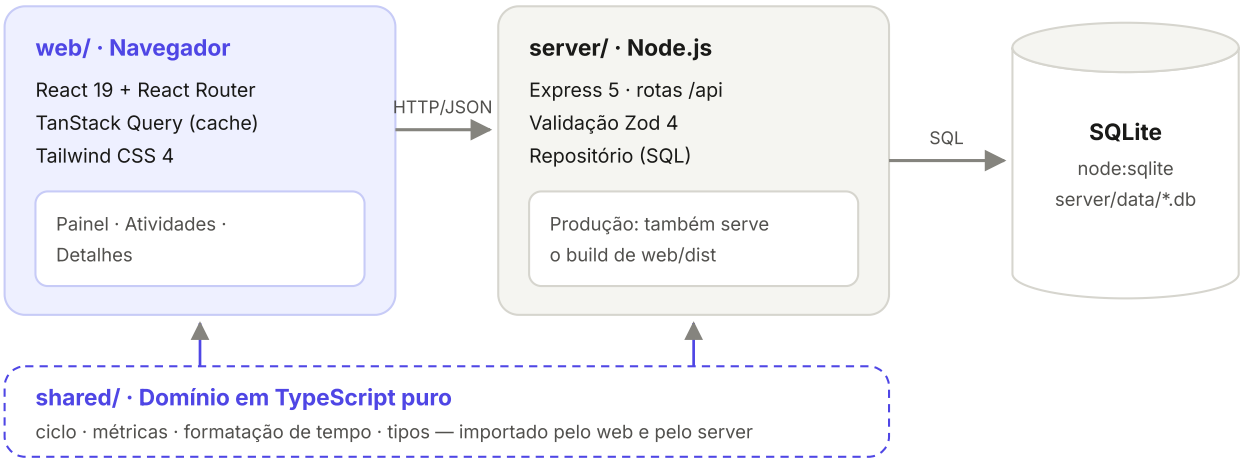


Figura 1 — Componentes e fluxo de dados. Em desenvolvimento, o Vite (porta 5173) encaminha `/api` para o Express (porta 3001).

3.1 Fluxo de uma requisição

1. A tela chama um *hook* do TanStack Query (ex.: `useCycle(data)`), que executa `fetch('/api/cycles/:data')`.
2. O Express valida os parâmetros com Zod e consulta o repositório, que executa SQL preparado no SQLite.
3. As atividades do ciclo são agregadas por `buildCycleSummary` (pacote `shared`) e devolvidas como JSON.
4. Mutações (criar, editar, excluir, alterar meta) invalidam as consultas `cycle` e `activities`; a interface é atualizada automaticamente.

3.2 Decisões técnicas

DECISÃO	MOTIVAÇÃO
Tempo em minutos inteiros	Comparações exatas (`330 === 330`). Elimina o arredondamento `ROUND(x·1440)` que a planilha usava para contornar ponto flutuante.
Domínio no pacote `shared`	Uma única implementação das regras para servidor e interface, testável sem I/O.
`node:sqlite`	Driver SQLite embutido no Node.js: sem dependência nativa para compilar (relevante no Windows) e com API síncrona simples.
TypeScript sem build no servidor	O Node executa `.ts` diretamente (*type stripping*). O código segue `erasableSyntaxOnly`: sem `enum`, `namespace` ou *parameter properties*.
Duração derivada	Persistir apenas início e fim evita inconsistência entre colunas.
Ciclo derivado da data	Dispensa tabela de semanas e qualquer rotina de “abertura” de ciclo.
Estado na URL	A data de referência (`?data=`) sobrevive a recarregamentos e permite compartilhar links para um dia específico.

3.3 Estrutura de diretórios

```
acompanhamento_diario/
├── package.json            workspaces e scripts raiz (dev, test, typecheck, build, start)
├── tsconfig.base.json      configuração TypeScript compartilhada
├── docs/                   plano de ação, esta documentação (PDF) e sua fonte
├── shared/
│   ├── src/  cycle.ts · metrics.ts · time.ts · types.ts · index.ts
│   └── test/ rules.test.ts
├── server/
│   ├── src/  index.ts · app.ts · repo.ts · db.ts
│   ├── test/ api.test.ts
│   └── data/               banco SQLite local (ignorado pelo git)
└── web/
    ├── index.html · vite.config.ts
    └── src/
        ├── main.tsx · hooks.ts · index.css
        ├── api/client.ts
        ├── pages/        Dashboard.tsx · Activities.tsx · Details.tsx
        └── components/   gráficos, formulário, badges, navegação, tema, toasts
```

4 Pacote `shared`

Funções puras e tipos do domínio. O pacote não depende de nada externo e é exportado como `@acomp/shared`.

4.1 `time.ts` — tempo

FUNÇÃO	COMPORTAMENTO
`parseHM(text): number | null`	`"08:45"` → `525`. Retorna `null` fora do formato `h:mm` / `hh:mm`.
`formatHM(min): string`	`525` → `"08:45"`. Acima de 24h não reinicia: `1650` → `"27:30"`.
`formatDuration(min): string`	Formato de leitura: `330` → `"5h30"`, `480` → `"8h"`, `45` → `"45min"`.
`formatDiff(diff): string`	`0` → `"00:00"`, `15` → `"+00:15"`, `-70` → `"-01:10"`.

4.2 `cycle.ts` — datas e ciclo

FUNÇÃO	COMPORTAMENTO
`getCycle(ref): Cycle`	Retorna `{ start, end, workdays[5] }` do ciclo que contém `ref`.
`isWorkday(iso)`	`true` de segunda a sexta.
`isValidISODate(text)`	Valida formato e existência da data (rejeita `2026-02-30`).
`todayISO()`	Data de hoje no fuso local.
`addDays`, `weekdayOf`	Aritmética de datas em UTC.
`remainingWorkdays(cycle, today)`	Dias úteis do ciclo posteriores a `today`.
`formatDayMonth(iso)`	`"2026-10-23"` → `"23/10"`.

4.3 `types.ts` — tipos e constantes

```
type ActivityStatus = 'concluido' | 'nao_concluido' | 'pendente_aprovacao' | 'nao_iniciado';

interface Activity {
  id: number; date: ISODate; startMinute: number; endMinute: number;
  project: string; description: string; notes: string; status: ActivityStatus;
}
type ActivityInput = Omit<Activity, 'id'>;

GOAL_PRESETS = [{ label: '5h30', minutes: 330 }, { label: '8h48', minutes: 528 }]
DEFAULT_DAILY_GOAL = 330      WORKDAYS_PER_CYCLE = 5
```

4.4 `metrics.ts` — métricas

FUNÇÃO	COMPORTAMENTO
`dayMetrics(date, activities, goal)`	Calcula `DayMetrics` de um dia (seção 2.4). Ignora atividades de outras datas.
`weekMetrics(days, dailyGoal)`	Agrega os dias em `WeekMetrics` (seção 2.5).
`buildCycleSummary(ref, activities, goal)`	Monta o `CycleSummary` completo: limites do ciclo, meta, 5 dias e totais.
`durationOf(activity)`	`max(0, end - start)`.

```
interface DayMetrics {
  date; weekdayShort; weekdayLong; goal; minutes; diff; diffLabel;
  activityCount; conformity: 'conforme' | 'nao_conforme' | 'pendente';
  exceeded: boolean; diagnosis: string;
}
interface WeekMetrics {
  goal; total; diff; diffLabel; activityCount; conformDays; exceededDays;
  fillPercent; conformityLabel; diagnosis;
}
interface CycleSummary { start; end; dailyGoal; days: DayMetrics[]; week: WeekMetrics }
```

5 Back-end

Servidor Express organizado em quatro módulos: conexão, repositório, aplicação HTTP e ponto de entrada.

5.1 `db.ts` — esquema

Abre o banco, ativa o modo **WAL** e aplica o esquema de forma idempotente (`CREATE … IF NOT EXISTS`). Restrições de integridade também são impostas pelo SQLite (`CHECK`).

```
CREATE TABLE activities (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  date         TEXT    NOT NULL,                 -- 'AAAA-MM-DD'
  start_minute INTEGER NOT NULL,
  end_minute   INTEGER NOT NULL,
  project      TEXT    NOT NULL,
  description  TEXT    NOT NULL DEFAULT '',
  notes        TEXT    NOT NULL DEFAULT '',
  status       TEXT    NOT NULL CHECK (status IN
               ('concluido','nao_concluido','pendente_aprovacao','nao_iniciado')),
  created_at   TEXT    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   TEXT    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK (end_minute > start_minute)
);
CREATE INDEX idx_activities_date ON activities(date);

CREATE TABLE cycle_goals (cycle_start TEXT PRIMARY KEY, daily_goal_minutes INTEGER NOT NULL);
CREATE TABLE settings    (key TEXT PRIMARY KEY, value TEXT NOT NULL);
```

5.2 `repo.ts` — repositório

Encapsula todo o SQL em *prepared statements* e converte as linhas (*snake_case*) para objetos do domínio (*camelCase*).

OPERAÇÃO	DESCRIÇÃO
`listActivities(from, to)`	Atividades no intervalo, ordenadas por data, início e id.
`getActivity` · `createActivity` · `updateActivity` · `deleteActivity`	CRUD por id. A atualização também grava `updated_at`.
`findOverlaps(a, ignoreId)`	Atividades do mesmo dia com `start < a.end` e `end > a.start`. Intervalos que apenas se encostam não contam como sobreposição.
`goalForCycle(cycleStart)`	Meta do ciclo ou, na ausência, a meta padrão.
`setCycleGoal(start, min, makeDefault)`	Grava a meta em transação. Com `makeDefault`: congela os ciclos anteriores, remove metas de ciclos posteriores e atualiza o padrão.

5.3 `app.ts` — aplicação HTTP

- `createApp({ repo, today?, staticDir? })` cria a instância Express. `today` é injetável para testes determinísticos.
- Corpos e parâmetros são validados com **Zod**. A primeira mensagem de erro é devolvida em `error`, e a lista completa em `issues`.
- JSON malformado gera `400`, rotas `/api` inexistentes geram `404` e exceções não tratadas geram `500` com registro no console.
- Quando `web/dist` existe, o servidor serve os arquivos estáticos e devolve `index.html` para as demais rotas (*SPA fallback*).

5.4 `index.ts` — ponto de entrada

VARIÁVEL	PADRÃO	USO
`PORT`	`3001`	Porta HTTP do servidor.
`DB_PATH`	`server/data/acompanhamento.db`	Arquivo SQLite. Aceita `:memory:`.

6 Referência da API REST

Base: `/api`. Todas as requisições e respostas usam `application/json`. Horários são expressos em minutos desde 00:00.

GET `/api/cycles/:date`

Resumo do ciclo que contém `:date`: os 5 dias úteis com métricas e os totais da semana.

STATUS	CORPO
`200`	`CycleSummary`
`400`	Data inválida

```
{ "start": "2026-10-23", "end": "2026-10-29", "dailyGoal": 330,
  "days": [ { "date": "2026-10-26", "weekdayShort": "Seg", "minutes": 345, "diff": 15,
              "diffLabel": "+00:15", "activityCount": 4, "conformity": "nao_conforme",
              "exceeded": true, "diagnosis": "Excedeu a meta de 5h30 em 00:15. Reajuste o apontamento", … }, … ],
  "week": { "goal": 1650, "total": 675, "fillPercent": 40, "conformDays": 1, "exceededDays": 1, … } }
```

PUT `/api/cycles/:date/goal`

Define a meta diária do ciclo de `:date`. Em ciclo atual ou futuro, ela também se torna o padrão dos próximos ciclos.

```
// requisição                          // resposta 200
{ "dailyGoalMinutes": 528 }            { "cycleStart": "2026-10-23", "dailyGoalMinutes": 528,
                                         "appliesToNextCycles": true }
```

STATUS	CONDIÇÃO
`400`	Data inválida, ou meta fora de 1–1440 ou não inteira

GET `/api/activities?date=AAAA-MM-DD`

Atividades de um dia, ordenadas por horário de início. Responde `200` com `Activity[]`, ou `400` se a data for inválida.

POST `/api/activities` · PUT `/api/activities/:id`

Cria ou substitui uma atividade. O corpo é um `ActivityInput` (seção 2.3). A resposta inclui avisos de sobreposição de horário.

```
// requisição
{ "date": "2026-10-26", "startMinute": 480, "endMinute": 525, "project": "Projeto Exemplo",
  "description": "Leitura da documentação", "notes": "", "status": "concluido" }

// resposta 201 (POST) / 200 (PUT)
{ "activity": { "id": 7, …campos… },
  "warnings": ["Sobrepõe \"Suporte\" (08:30-09:00)."] }
```

STATUS	CONDIÇÃO
`400`	Falha de validação: fim ≤ início, fim de semana, status inválido, projeto vazio, JSON malformado
`404`	`PUT` com `id` inexistente

DELETE `/api/activities/:id`

Remove a atividade. Responde `204` sem corpo, ou `404` se o `id` não existir.

Formato de erro

```
HTTP/1.1 400 Bad Request
{ "error": "O horário de fim deve ser depois do início.",
  "issues": [ { "path": ["endMinute"], "message": "…", "code": "custom" } ] }
```

7 Front-end

SPA em React 19 com Vite. O estado de servidor fica no TanStack Query e o estado de navegação, na URL.

7.1 Rotas e telas

ROTA	TELA	CONTEÚDO
`/`	Painel	% preenchido, KPIs (meta semanal, total realizado, dias conformes), alerta de irregularidade, status do ciclo, gráfico por dia útil, seletor de meta, "Registrar atividades" e "Mostrar mais".
`/atividades`	Atividades	Abas Sex–Qui com progresso, resumo do dia, lista com edição e exclusão confirmada, formulário com duração em tempo real.
`/detalhes`	Detalhamento	Tabela equivalente à da planilha, com linha de total geral.

Todas as telas aceitam `?data=AAAA-MM-DD`. Sem o parâmetro, vale a data de hoje. Em `/atividades`, datas de fim de semana são resolvidas para o último dia útil do ciclo.

7.2 Estado e dados (`api/client.ts`)

HOOK	QUERY KEY / EFEITO	USO
`useCycle(date)`	`['cycle', inicioDoCiclo]`	Resumo do ciclo (todas as telas)
`useDayActivities(date)`	`['activities', date]`	Lista do dia
`useSaveActivity()`	invalida `cycle` e `activities`	Cria (sem `id`) ou edita
`useDeleteActivity()`	invalida `cycle` e `activities`	Exclusão
`useSetGoal()`	invalida `cycle` e `activities`	Altera a meta do ciclo

`QueryClient`: `staleTime` de 30 s, 1 nova tentativa, revalidação ao focar a janela. Falhas de rede viram `ApiError` com mensagem em português.

7.3 Componentes principais

COMPONENTE	RESPONSABILIDADE
`Layout`	Cabeçalho fixo, navegação que preserva `?data=`, botão de tema.
`CycleNav`	Navegação entre ciclos (±7 dias) e atalho "Hoje".
`GoalSelector`	Controle segmentado 5h30 / 8h48 / Outro, com campo `hh:mm`.
`ProgressRing`	Anel SVG do % preenchido: índigo em andamento, verde quando completo, vermelho quando excedido.
`DailyChart`	Barras por dia útil contra a linha de meta, com tooltip e clique para abrir o dia.
`IrregularAlert`	Lista os dias acima da meta, com link direto para reajuste.
`CycleBanner`	Contagem até quinta, aviso de dia de apontamento e fechamento de ciclos passados.
`ActivityForm`	Formulário de criação/edição. Sugere o início a partir do fim da última atividade do dia.
`Badges`	`dayTone` classifica o dia em *conforme*, *abaixo*, *acima* ou *pendente*; badges de conformidade e de status.
`Toast`	Notificações de sucesso, aviso e erro (região `aria-live`).

7.4 Linguagem visual e tema

- **Tokens de design** em variáveis CSS (`index.css`), expostos ao Tailwind via `@theme inline` (ex.: `bg-surface`, `text-ink-2`).
- **Cores de status reservadas:** verde (conforme), âmbar (abaixo da meta), vermelho (acima da meta/irregular) e cinza (pendente). Toda cor de status vem acompanhada de ícone e rótulo, nunca sozinha.
- **Tema:** segue `prefers-color-scheme` até o usuário escolher um. O `ThemeToggle` grava `data-theme` em `<html>` e persiste a escolha em `localStorage['tema']`. Um script inline em `index.html` aplica o tema salvo antes da primeira pintura, evitando o efeito de piscar.
- **Acessibilidade:** rótulos ARIA nos controles de ícone, `role="tablist"` nas abas de dias, foco visível e respeito a `prefers-reduced-motion`.

8 Execução e operação

A aplicação roda localmente, sem serviços externos. O único requisito é o Node.js.

8.1 Requisitos

- **Node.js ≥ 23.6**, necessário para executar TypeScript nativamente e para o módulo `node:sqlite`. O projeto foi desenvolvido no Node 26.
- npm 10 ou superior.

8.2 Instalação e uso

```
git clone https://github.com/peubibytes/acompanhamento_diario.git
cd acompanhamento_diario
npm install

npm run dev      # desenvolvimento: API em :3001 e interface em http://localhost:5173
npm start        # uso diário: build da interface + servidor único em http://localhost:3001
```

8.3 Scripts

SCRIPT	AÇÃO
`npm run dev`	Executa em paralelo (`concurrently`) o servidor com `node --watch` e o Vite com HMR.
`npm test`	Testes de domínio e de API com `node --test`.
`npm run typecheck`	`tsc` em `shared`, `server` e `web` (modo `strict`).
`npm run build`	Build de produção da interface em `web/dist`.
`npm start`	Build seguido do servidor, que entrega API e interface na mesma porta.
`npm run docs:pdf`	Regenera este PDF a partir de `docs/fonte/documentacao.html` (requer Microsoft Edge ou Google Chrome).

8.4 Dados e backup

Todos os dados ficam em `server/data/acompanhamento.db`, junto com os arquivos `-wal` e `-shm` do modo WAL. O diretório é ignorado pelo git. Para backup, copie o arquivo com o servidor parado. Para mover os dados, use `DB_PATH`.

Privacidade. O repositório é público. Registros reais de trabalho (o banco local e a planilha de referência) nunca são versionados: `server/data/` e `*.xlsx` estão no `.gitignore`, e os exemplos de código usam dados fictícios.

9 Qualidade

Testes automatizados sobre as regras e o contrato HTTP, com tipagem estrita em todo o monorepo.

9.1 Testes

SUÍTE	COBERTURA
`shared/test/rules.test.ts`	Conversão e formatação de tempo; ciclo sexta→quinta, incluindo viradas de mês e de ano; métricas diárias com o cenário de exemplo da planilha; conformidade exata; excesso com metas de 8h48 e personalizada; % preenchido com arredondamento para baixo; semana 100% conforme.
`server/test/api.test.ts`	Servidor real sobre banco `:memory:`: CRUD completo, validações (horário, fim de semana, status, projeto), avisos de sobreposição, resumo do ciclo, regras de meta para ciclo atual, futuro e passado, JSON malformado.

Total atual: **23 testes**, executados com o *test runner* nativo do Node.js, sem dependências adicionais.

9.2 Convenções

- TypeScript em modo `strict`, com `verbatimModuleSyntax` e `erasableSyntaxOnly`; imports com extensão `.ts` no servidor e no domínio.
- Identificadores de código em inglês; textos de interface, mensagens de erro e comentários em português.
- Regras de negócio apenas em `shared`; o servidor valida e persiste, e a interface apenas apresenta.
- Commits com mensagens descritivas em português.

10 Roadmap

Evoluções previstas, em ordem aproximada de prioridade.

ITEM	DESCRIÇÃO
Integração com o Azure DevOps	Associar atividades a work items e enviar os apontamentos do ciclo às quintas-feiras.
Autocompletar projetos	Sugerir projetos já usados no ciclo ou em ciclos anteriores.
Duplicar atividade	Copiar atividades recorrentes de outro dia.
Exportação	Exportar o ciclo em CSV/XLSX.
Feriados	Marcar dias úteis sem expediente e ajustar a meta semanal.